Limbajul PL/SQL

Inroducere; Variabile, expresii si operatori

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Limbajul PL/SQL

- PL/SQL (Procedural Language/SQL) este o extensie procedurală a limbajului SQL pentru SGBD Oracle (extensie proprietară).
- Apariţia sa este datorată faptului că limbajul SQL este un limbaj de cereri şi nu un limbaj de programare.
- Unitatea de execuţie este **blocul** care conţine atât cereri SQL cât şi instrucţiuni PL/SQL. Ele sunt executate astfel:
  - Cererile SQL sunt trimise pentru a fi executate de serverul Oracle. Rezultatele acestora pot fi folosite apoi în instrucţiunile PL/SQL.
  - Instrucţiunile PL/SQL sunt executate de **Executorul de Instrucţiuni Procedurale** (Procedural Statement Executor) aflat în **Motorul PL/SQL** (PL/SQL Engine).

Limbajul PL/SQL

- Motorul PL/SQL este prezent în:
  - Serverul Oracle
  - Unelte Oracle care conţin propriul lor motor PL/SQL. Ele nu folosesc motorul PL/SQL integrat în serverul Oracle decât pentru porţiunile de bloc reprezentând cereri SQL, minimizând astfel încărcarea serverului.

Limbajul PL/SQL

- PL/SQL este folosit pentru a scrie:
  - Blocuri anonime (Anonymous blocks)
  - Proceduri şi funcţii stocate (Stored procedures/functions)
  - Proceduri şi funcţii folosite în aplicaţii (Application procedures/functions)
  - Declanşatori ataşaţi bazei de date (Database triggers)
  - Declanşatori folosiţi în aplicaţii (Application triggers)
  - Pachete (Packaged procedures)

Avantajele folosirii PL/SQL

- Integrare
- Îmbunătăţirea performanţelor
- Dezvoltare modulară a programelor
- Portabilitate
- Este procedural

Avantajele folosirii PL/SQL

Integrare:

- Prezenţa motorului PL/SQL atât în serverul Oracle cât şi în uneltele Oracle
- Bibliotecile comune (shared libraries) cu proceduri şi funcţii scrise în PL/SQL pot fi utilizate de toate aplicaţiile dezvoltate cu Developer/2000 şi pot fi accesate local sau la distanţă.

Avantajele folosirii PL/SQL

- **Integrare:**

- Tipurile de date din SQL pot fi folosite şi în PL/SQL, dar există anumite diferenţe privind dimensiunile maxime pentru unele dintre ele. Aceste diferenţe sunt discutate în capitolele următoare
- În cazul uneltelor Oracle având încorporat motor PL/SQL, executorul de instrucţiuni procedurale procesează date locale aplicaţiei (client) reducând încărcarea serverului şi necesarul de memorie pentru acesta.

Avantajele folosirii PL/SQL

- **Îmbunătăţirea performanţelor:**

- Un bloc care poate conţine mai multe instrucţiuni SQL este trimis o singură dată serverului reducându-se traficul pe reţea.
- Mărirea performanţelor uneltelor Oracle prin adăugarea facilităţilor procedurale ale PL/SQL.
- Procedurile şi funcţiile care fac parte din aplicaţii dezvoltate cu Developer/2000 sunt diferite de cele stocate în baza de date deşi au aceeaşi structură. În acelaşi timp aceste aplicaţii pot folosi subprogramele stocate în baza de date gestionată de serverul Oracle.

Avantajele folosirii PL/SQL

Dezvoltare modulară a programelor:

- Un program PL/SQL (PL/SQL unit) este format din unul sau mai multe blocuri – proceduri, funcţii şi blocuri anonime. Un bloc poate conţine în interiorul său alte blocuri.
- Un program complex poate fi divizat în module logice distincte implementate prin blocuri.
- Codul reutilizabil PL/SQL poate fi memorat în librării comune în cazul aplicaţiilor Developer/2000 sau stocat pe server, în care caz poate fi folosit de orice aplicaţie care interacţionează cu baza de date Oracle.

Avantajele folosirii PL/SQL

Portabilitate:

- Este acelaşi pentru orice platformă Oracle, indiferent de sistemul de operare sau platforma hardware folosită.
- Programele scrise în PL/SQL pot fi folosite în diverse medii (server, aplicaţii).

Avantajele folosirii PL/SQL

Limbaj procedural:

- Se pot defini variabile, cursori, excepţii (tratarea erorilor). Variabilele pot fi scalare, referinţă, compuse sau de tip obiect mare (large object - LOB). Acestea pot fi definite şi dinamic pe baza structurii tabelelor din baza de date şi a coloanelor acestora.
- Există instrucţiuni de decizie, cicluri, parcurgere (pentru liniile unui rezultat de cerere SQL de tip SELECT).

Avantajele folosirii PL/SQL

Limbaj procedural:

- Permite tratarea erorilor apărute în execuţia cererilor SQL de către serverul Oracle (erorile se numesc în acest context **excepţii**) prin rutine asociate diverselor tipuri de erori care pot apare.
- Se pot declara de către utilizator condiţii suplimentare de eroare şi asocia rutine de tratare.

CE PUTEM SCRIE IN PL/SQL?

- Blocuri anonime (Anonymous blocks)
- Proceduri şi funcţii stocate (Stored procedures/functions)
- Proceduri şi funcţii folosite în aplicaţii (Application procedures/functions)
- Declanşatori ataşaţi bazei de date (Database triggers)
- Declanşatori folosiţi în aplicaţii (Application triggers)
- Pachete (Packaged procedures)

CUM ARATA UN BLOC?

- Un bloc anonim are urmatoarea structura:

```
DECLARE      -- opţional
declaraţii de variabile, cursori, excepţii definite de utilizator
BEGIN        -- obligatoriu
cereri SQL
instrucţiuni PL/SQL
EXCEPTION    -- opţional
acţiuni executate în caz de ridicare excepţii
END;         -- obligatoriu
```

Structura unui bloc

Între DECLARE şi BEGIN:

- Este zona de declaraţii a blocului. Sunt declarate în principal variabile, cursori, excepţii utilizator necesare blocului, dar şi alte elemente (tipuri, proceduri, funcţii, etc.).
- Această parte este opţională, un bloc putând începe direct cu cuvântul cheie BEGIN (care este obligatoriu).
- Tipurile de date din SQL pot fi folosite (cu unele diferenţe) şi în PL/SQL.

Structura unui bloc

Între BEGIN şi EXCEPTION:

- Corpul blocului, format din cereri SQL şi instrucţiuni PL/SQL care descriu procesarea datelor în cadrul acestuia.
- Cuvântul cheie EXCEPTION poate lipsi, în care caz blocul se termină cu END.

Structura unui bloc

Între EXCEPTION şi END:

- Acţiuni executate în caz de excepţie (în principal tratarea erorilor).
- Este o parte opţională a unui bloc (dar cuvântul cheie END care marchează sfârşit de bloc este obligatoriu).

Exemplu

```
DECLARE
v_nume    VARCHAR2(10);
BEGIN
SELECT NUME
INTO v_nume
FROM SPEC
WHERE CODS=11;
DBMS_OUTPUT.PUT_LINE('Specializarea: '|| v_nume);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('A aparut o exceptie');
END;
```

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- Fiecare cerere SQL şi instrucţiune PL/SQL se încheie cu punct şi virgulă (;)
- DECLARE, BEGIN şi EXCEPTION nu se termină cu punct şi virgulă
- După END se pune punct şi virgulă
- Terminarea editarii blocului se face cu punct (**.**)
- Rularea unui bloc se face cu **/** sau **r** (run) la promptul SQL>. În sistemul de operare Windows rularea se poate comanda şi prin File → Run.

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- Ultimul bloc executat poate fi editat. În acest caz marcajul de terminare bloc este ca şi în cazul cererilor SQL un slash (**/**) aflat singur pe ultima linie a fişierului.
- Editarea se poate iniţia prin comanda **ed** la promptul SQL> sau pe calea:

Edit → *Editor* → *Invoke Editor*

Executia blocului

În cazul execuţiei un bloc din SQL*Plus:

- În caz de rulare cu succes, după eventualele mesaje tipărite de bloc apare:

  `PL/SQL procedure successfully completed`

- În caz de rulare cu eroare netratată, se va afişa codul şi mesajul de eroare.

Executia blocului

- În exemplul de mai sus a fost folosită pentru afişarea mesajului 'A aparut o exceptie' procedura DBMS_OUTPUT.PUT_LINE.
- Aceasta face parte din pachetul DBMS_OUTPUT pus la dispoziţie de sistemul Oracle şi va fi utilizată şi în capitolele următoare. Sintaxa este:

```
DBMS_OUTPUT.PUT_LINE(expresie)
```

Executia blocului

- Procedura evaluează valoarea expresiei, o converteşte la şir de caractere şi o tipăreşte.
- Cum implicit SQL*Plus nu afişează mesajele tipărite în cadrul blocului, pentru a le putea vedea opţiunea SQL*PLUS ***serveroutput*** trebuie trecută pe ON prin comanda:
- **SQL> SET SERVEROUTPUT ON**

Variabile în PL/SQL

- Un bloc PL/SQL poate avea propriile sale variabile.
- În continuare sunt prezentate elementele de bază referitoare la declararea şi folosirea acestora.

Categorii de variabile

- Variabile PL/SQL
- Variabile externe

Variabile PL/SQL

Scalare:

- Aceste variabile pot conţine o singură valoare. Tipurile principale corespund celor care se pot asocia coloanelor unei tabele Oracle.

Compuse:

- Conţin mai multe valori, de exemplu o înregistrare (linie) din rezultatul unei cereri SQL

Variabile PL/SQL

Referinţă:

- Conţin pointeri (referinţe) către alte elemente de program. Nu sunt tratate în acest curs (cu excepţia tipului REF CURSOR)

LOB (obiecte mari):

- Acestea conţin valori numite locatori (locators) care specifică locaţia unor obiecte mari (imagini de exemplu).

Variabile externe

- Variabile care provin din limbajul gazdă în care este scrisă aplicaţia (C, PHP, etc).
- Câmpuri ale unei forme în Oracle Forms
- Variabile SQL*Plus

Declararea variabilelor

- Sintaxa unei declaraţii de variabile este:

```
Identificator [CONSTANT]
tipdedate [NOT NULL] [:= |
DEFAULT expresie]
```

Unde **Identificator :**

- Numele variabilei (maxim 30 caractere).
- Se aplică aceleaşi reguli ca la orice alt obiect SQL (tabele, coloane, etc).

Declararea variabilelor

- Două obiecte pot avea acelaşi nume doar dacă sunt declarate în blocuri diferite. În acest caz în fiecare bloc poate fi folosit doar cel declarat local.
- Este de dorit ca variabilele să nu aibă aceleaşi nume cu coloanele sau tabelele din baza de date. În caz contrar Oracle consideră numele ca fiind cel al coloanei sau tabelei.

Declararea variabilelor

[CONSTANT]

- Variabila nu-şi poate schimba valoarea. Constantele trebuiesc iniţializate la declarare.

tipdedate

- Tipul variabilei (un tip scalar, compus, referinţă sau LOB).

[NOT NULL]

- Variabila nu poate fi nulă. Aceste variabile trebuiesc iniţializate la declarare.

Declararea variabilelor

[:= expresie]

- Iniţializare cu o expresie. Variabilele neiniţializate conţin iniţial valoarea NULL.

[DEFAULT expresie]

- Valoare implicită dată de o expresie (iniţializează variabila).

Exemplu

```
DECLARE
v_nume  VARCHAR2(10)  := 'ION';
v_datanasterii     DATE;
v_cods  NUMBER(2) NOT NULL := 21;
v_suma  CONSTANT NUMBER := 100;
.....
```

Asignarea variabilelor

- Instrucţiunea de atribuire este **:=** (ca în limbajul Pascal).
- Caracterul **=** este folosit în PL/SQL ca operator de comparaţie (egal cu).
- Sintaxa atribuirii este:

```
Identificator := expresie;
```

Exemple

```
v_nume := 'ION';
v_datanasterii := '12-APR-89';
v_data2 := to_date('14-04-79', 'DD-MM-YY');
v_numar := v_n1 + v_n2 * 1230;
```

Tipuri de date folosite

- Tipuri numerice scalare
- Tipuri scalare caracter/binare
- Tipuri LOB (Large Objects)
- Tipul BOOLEAN
- Tipuri Data, Timp si Interval
- Variabile definite cu %TYPE
- Variabile externe

Tipuri numerice scalare

- **NUMBER[(număr de cifre, număr de zecimale)]:** Număr întreg sau real. Fără parametri sau cu 2 parametri: număr real. Cu un parametru: număr întreg. Magnitudine: 10^{-130} - 10^{125} (+/-).
- Subtipuri ale acestui tip sunt:
  - DEC, DECIMAL, NUMERIC – numere în virgulă fixă cu 38 de cifre semnificative.
  - DOUBLE PRECISION, FLOAT: numere în virgulă mobilă cu 38 de cifre semnificative.
  - REAL: numere în virgulă fixă cu 18 cifre semnificative.
  - INTEGER, INT, SMALLINT: numere întregi cu maxim 38 de cifre semnificative.

Tipuri numerice scalare

- **BINARY_INTEGER:** Întregi între -2^{31} şi 2^{31}.
  Subtipuri ale acestuia sunt:
  - NATURAL şi POSITIVE pentru întregi non-negativi sau pozitivi.
  - NATURALN şi POSITIVEN ca mai sus dar nu pot avea valori nule
  - SIGNTYPE: doar valorile -1, 0 şi 1.

Tipuri numerice scalare

- **PLS_INTEGER:** Întregi între -2^{31} şi 2^{31}. E asemănător cu tipul anterior dar operaţiile sunt mai rapide.
- În plus, în caz de depăşire se ridică o excepţie (la BINARY_INTEGER şi NUMBER nu se întâmplă asta).
- Este recomandat în aplicaţii noi în loc de BINARY_INTEGER.

Tipuri scalare caracter/binare

CHAR[(lungime_maximă [CHAR | BYTE])]:

- Şir de caractere de lungime fixă (maxim 32737 octeţi, implicit 1). Opţional CHAR | BYTE spune în ce este exprimată lungimea maximă (un caracter putând fi memorat pe mai mulţi octeţi în funcţie de setul de caractere folosit).
- Un subtip al acestui tip este CHARACTER, identic cu tipul de bază (introdus pentru compatibilitatea cu alte sisteme).
- **Notă:** acest tip diferă de tipul CHAR pentru coloane care poate avea maxim 2000 de octeţi.

Tipuri scalare caracter/binare

VARCHAR2(lungime_maximă [CHAR | BYTE]):

- Şir de caractere de lungime variabilă, maxim 32737 octeţi. Aceleaşi observaţii ca mai sus în legatură cu CHAR | BYTE.
- Subtipuri ale acestui tip sunt: STRING şi VARCHAR, identice cu tipul de bază (introduse pentru compatibilitatea cu alte sisteme).
- **Notă:** acest tip diferă de tipul VARCHAR2 pentru coloane care poate avea maxim 4000 de octeţi.

Tipuri scalare caracter/binare

LONG

- Similar cu VARCHAR2 dar lungimea maximă este de 32760 de octeţi. Începând cu Oracle 9i acest tip este compatibil cu CLOB şi se recomandă folosirea acestuia din urmă.
- **Notă:** acest tip diferă de tipul LONG pentru coloane care poate avea maxim 2^{31} octeţi (2 GB).

Tipuri scalare caracter/binare

LONG RAW:

- Similar cu LONG dar se pot stoca şi date binare (nu doar caractere). În plus, PL/SQL nu interpretează date de acest tip.
- Începând cu Oracle 9i acest tip este compatibil cu BLOB şi se recomandă folosirea acestuia din urmă.
- **Notă:** acest tip diferă de tipul LONG RAW pentru coloane care poate avea maxim 2^{31} octeţi.

Tipuri scalare caracter/binare

RAW(dimensiune_maximă)

- Similar cu VARCHAR2 conţine date binare. Dimensiunea maximă este de 32767 octeţi. PL/SQL nu interpretează astfel de date.
- **Notă:** acest tip diferă de tipul LONG RAW pentru coloane care poate avea maxim 2000 de octeţi.

Tipuri scalare caracter/binare

ROWID

- Poate stoca un identificator fizic de linie într-o tabelă. Pentru conversia la/de la şir de caractere (18 caractere) se pot folosi funcţiile SQL ROWIDTOCHAR respectiv CHARTOROWID.

UROWID

- (Universal ROWID) Poate stoca un identificator logic de linie într-o tabelă, indexată sau nu precum şi un identificator de linie extern (non-Oracle). Nu este necesară folosirea funcţiilor de conversie la/de la şir de caractere (conversie automată).

Tipuri scalare caracter/binare

NCHAR(dimensiune_maximă)

- Echivalent cu CHAR dar pentru a stoca date în setul de caractere naţional specificat la crearea bazei (un caracter pe 2-3 octeţi).

NVARCHAR2(dimensiune maximă)

- Echivalent cu VARCHAR2 dar pentru a stoca date în setul de caractere naţional specificat la crearea bazei (un caracter pe 2-3 octeţi).

Tipuri LOB

- Aceste tipuri permit stocarea de date nestructurate (text, imagini, video, audio) de dimensiune maximă 4 Gbytes.
- Este posibil accesul direct la conţinut (la LONG doar acces secvenţial).
- Obiectele de acest tip conţin de fapt referinţe (locatori) pentru date, datele propriu-zise fiind stocate fie în baza de date fie în fişiere externe din sistemul de operare.
- Sunt de următoarele tipuri:

Tipuri LOB

BFILE:

- Date binare (până la 4GB) stocate în fişiere externe, din sistemul de operare. Sunt date read/only (nu se pot modifica). Nu participă la tranzacţii şi nici la restaurarea bazei de date (ele nu sunt stocate în aceasta) şi nu pot fi replicate.

Tipuri LOB

BLOB:

- Pentru date binare de dimensiuni mari. Aparţin bazei de date, participă la tranzacţii şi se pot restaura cu baza de date, sunt replicabile. Dimensiunea maximă este de 4GB.

CLOB:

- Similar cu BLOB dar pentru date tip caracter şi nu binare. Aceleaşi caracteristici ca mai sus.

NCLOB:

- Similar cu CBLOB dar pentru şiruri de caractere din setul naţional setat la crearea bazei de date.

Tipul BOOLEAN

- Se pot stoca valori booleene: TRUE, FALSE şi NULL. Sunt permişi doar operatorii booleeni.
- **Notă:** acest tip nu există ca tip pentru coloanele unei tabele.

Tipuri Data, Timp, Interval

DATE:

- Dată calendaristică (zi, lună, an, oră, minut, secundă). Se pot folosi operatorii + număr şi - număr pentru a incrementa şi decrementa cu un număr de zile o dată.
- Diferenţa între două date dă numărul de zile dintre acestea (real, cu zecimale în cazul în care ora-minutul-secunda diferă).

Tipuri Data, Timp, Interval

TIMESTAMP[(precizie)]

- Extinde tipul DATE. Conţine dată calendaristică (zi, lună, an, oră, minut, secundă şi fracţiuni de secundă).
- Precizia (între 0 şi 9) specifică numărul de zecimale pentru fracţiunile de secundă. Valoarea implicită a preciziei este 6.

Exemplu:

```
DECLARE
t TIMESTAMP(4);
BEGIN
t := TIMESTAMP '1988-12-26 15:52:33.1592';
END;
```

Tipuri Data, Timp, Interval

TIMESTAMP [(precizie)] WITH TIME ZONE

- Extinde tipul TIMESTAMP conţinând şi o diferenţă între ora locală şi ora universală (GMT).

Exemplu:

```
DECLARE
t TIMESTAMP(4) WITH TIME ZONE;
BEGIN
t := TIMESTAMP '1988-12-26 15:52:33.1592 +2:00';
END;
```

- În acest caz diferenţa este de 2 ore.
- Două date de acest tip sunt identice dacă reprezintă aceeaşi dată în timp universal.

Tipuri Data, Timp, Interval

TIMESTAMP [(precizie)] WITH LOCAL TIME ZONE

- Similar cu tipul anterior dar la stocarea pe o coloană de tip DATE se face conversia la timpul local, deci diferenţa faţă de GMT nu este stocată.

Tipuri Data, Timp, Interval

INTERVAL YEAR [(precizie)] TO MONTH

- Se stochează intervale de ani şi luni. Precizia specifică numărul de cifre pentru an (între 0 şi 4, implicit 2). De exemplu, un interval de 2 ani şi 3 luni se poate stoca într-o variabilă de acest tip astfel:

Tipuri Data, Timp, Interval

```
DECLARE
perioada INTERVAL YEAR(1) TO MONTH;
BEGIN
perioada := INTERVAL '2-3' YEAR TO MONTH;
```

- sau:

```
perioada := '2-3';
```

- Se poate stoca numai un număr de ani sau de luni:

```
perioada := INTERVAL '2' YEAR;
perioada := INTERVAL '3' MONTH;
```

Tipuri Data, Timp, Interval

INTERVAL DAY [(precizie_zi)] TO SECOND [(precizie_sec)]

- Similar cu tipul anterior, dar pentru intervale de zile şi secunde.

Nota: Datele şi/sau intervalele se pot aduna, scădea, înmulţi rezultând date sau intervale, după caz:

Tipuri Data, Timp, Interval

```
Interval1 + Interval2 = Interval3
Interval1 - Interval2 = Interval3
Data1 + Interval = Data2
Data1 - Interval = Data2
Interval1 * numar = Interval3
Interval1 / numar = Interval3
```

Atributul %TYPE

- Putem defini tipul unei variabile în funcţie de tipul altei variabile sau a unei coloane de tabelă folosind construcţia:

`Variabila%TYPE` sau

`Tabela.Coloana%TYPE.`

- Exemplu:

```
DECLARE
   v_numar          NUMBER(5,2);
   v_altnumar            v_numar%TYPE;
   v_nume           stud.nume%TYPE;
```

- **Notă:** Dacă o variabilă este definită pe baza tipului unei coloane de tip NOT NULL ea nu moşteneşte acest atribut, putând conţine şi valoarea NULL.

Variabile externe (non PL/SQL)

- Acestea se prefixează cu două puncte (:). Exemplu:

```
:g_total := v_suma * 12;
```

- Astfel de variabile provin din mediul extern, de exemplu variabile din programele C, PHP, etc. din care se face acces la baza de date, câmpuri ale formelor sau variabile SQL*Plus.

Exemplu

```
<?php
// apelare script: prin formular sau direct (metoda GET)
  astfel:
// http://info.cs.pub.ro/~bdfr/prof/test6.php?matr=4311
// conectare si parse cerere
$c = oci_connect("scott","tiger");
$cerere = "begin
    select cods, nume into :cods, :nume from STUD
  where matr=".$_REQUEST['matr']."; ".
  "exception
          when others then
          :cods := 0; :nume := 'nu avem astfel de student';
          end;"; // atentie: nu trebuie uitat ; de dupa end!
echo "<h2>Cererea este: $cerere</h2>";
```

Exemplu

```
$stmt = oci_parse($c, $cerere);
// bind by name: se asociaza :ovariabila (din PL/SQL) cu
   $ovariabila (din PHP)
oci_bind_by_name($stmt, ":cods", $cods, 32);
oci_bind_by_name($stmt, ":nume", $nume, 32);
oci_execute($stmt,OCI_DEFAULT); // executie
if ($cods)
   echo $cods, " ",  $nume; // rezultatul va fi in $cods si
   $nume
else
   echo $nume;
?>
```

Identificatori

- Identificatorii obişnuiţi folosiţi în blocurile PL/SQL trebuie să îndeplinească următoarele condiţii:
  - Lungimea maximă este de 30 de caractere.
  - Trebuie să înceapă cu o literă.
  - Pot conţine litere, cifre şi caracterele _ $ #
- În cazul literelor nu contează dacă sunt mari sau mici. De exemplu identificatorul *NUME* este acelaşi cu *Nume* sau *nume*.
- Nu trebuie să fie identici cu numele unei coloane dintr-o tabelă a bazei de date
- Cuvintele rezervate pot fi folosite ca identificatori dar doar puse între ghilimele (exemplu "SELECT", "INSERT", "UPDATE", etc.).

Exemple

- Exemple de identificatori valizi:

```
v_nume          VARCHAR2(10);
v$nume#_        VARCHAR2(40);
v1_$            NUMBER;
```

- Exemple de identificatori invalizi:

```
1_nume          VARCHAR2(10);
v-nume&         VARCHAR2(40);
v/t=numar       NUMBER;
END             BOOLEAN;
```

Identificatori cu ghilimele

- Deşi nu sunt recomandaţi, în PL/SQL sunt permişi şi operatori care conţin orice caracter tipăribil (inclusiv spaţiu) dacă sunt puşi între ghilimele. Lungimea maximă este tot de 30 de caractere (exclusiv ghilimelele) şi în conţinutul lor nu poate apare caracterul ghilimele.
- Exemple de astfel de identificatori:

```
"1_nume"          VARCHAR2(10);
"v-nume& "        VARCHAR2(40);
"v/t=numar"       NUMBER;
"END"             BOOLEAN;
```

Identificatori cu ghilimele

- Ghilimelele sunt necesare şi atunci când în cererile SQL apar cuvinte rezervate PL/SQL dar care nu sunt rezervate în SQL.
- De exemplu, dacă o coloană a unei tabele se numeşte BEGIN (cuvânt rezervat doar în PL/SQL) atunci o cerere SQL care este parte a unui bloc PL/SQL de tipul:

```
SELECT EVENIMENT, BEGIN . . .
```

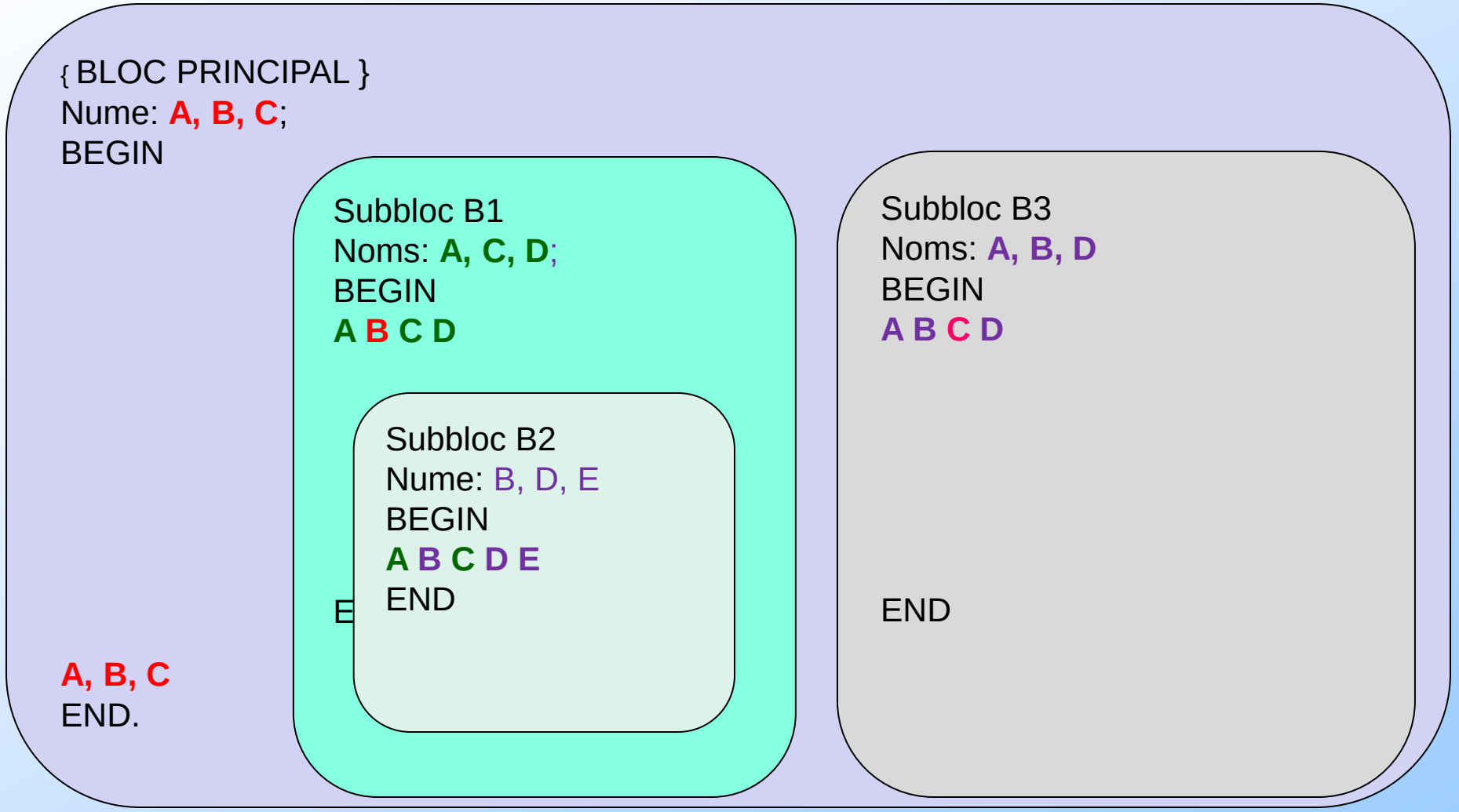
- va genera o eroare. Corect cererea trebuie scrisă astfel:

```
SELECT EVENIMENT, "BEGIN" . . .
```

Domeniul de valabilitate

- Un identificator definit într-un bloc este local acelui bloc şi global în toate subblocurile sale.
- În cazul în care un subbloc redefineşte un identificator, în subbloc poate fi folosit doar acesta din urmă.

Domeniul de valabilitate



Domeniul de valabilitate

- Exemplu:

```
DECLARE
  a NUMBER;
  b VARCHAR2(10);
BEGIN
  DECLARE
     a VARCHAR2(20);
  c NUMBER;
  BEGIN
  -- pot fi folositi a (VARCHAR2(20)), b si c
  END;
-- pot fi folositi a (NUMBER) si b
END;
```

Operatori PL/SQL

- În expresii se pot folosi operatori de comparaţie, aritmetici, logici, de concatenare precum şi cei patru operatori specifici SQL (IS NULL, LIKE, BETWEEN şi IN).
- Precedenţa operatorilor este cea din tabelul urmator:

Operatori PL/SQL

Operator	Semnificație
**	ridicare la putere
*, /	înmulțire, împărțire
+, -, ||	adunare, scădere, concatenare
=, <, >, <=, >=, <>, !=, ~=, ^=,IS NULL, LIKE, BETWEEN, IN	comparație
NOT	negație logică
AND	ȘI logic
OR	SAU logic

Operatori PL/SQL

- Pentru a preîntâmpina eventualele erori de semantică în scrierea expresiilor se recomandă folosirea parantezelor pentru a specifica explicit ordinea de efectuare a operaţiilor.
- De exemplu, următoarele două expresii logice sunt echivalente dar recomandată este a doua formă:

```
v_a > 1 and v_b < 10 or v_nr = 20
((v_a > 1) and (v_b < 10)) or (v_nr = 20)
```

Expresii

Constante (literali)

- Literalii pot fi:
- Numerici. Exemple: -12.9, 9E2 (= 9 * 10^2 = 900)
- Logici: Pot avea valoarea TRUE, FALSE şi NULL.
- Şir de caractere. Se pun între apostrofi. Exemple: 'A', 'Buna ziua', 'Don''t do that'. Între apostrofi poate fi orice caracter tipăribil. Includerea unui apostrof în şir se face prin dublarea sa, ca în ultimul exemplu
- Dată calendaristică. Se pun între apostrofi în formatul standard şi se foloseşte tipul lor generic în genul cast-ului din C (Oracle9).

Expresii

- Exemple de literali de tip dată:

```
DECLARE
d date := date '1988-05-25';
t timestamp :=
  timestamp '1988-06-20 11:00:00';
u timestamp with time zone :=
  timestamp '1988-06-20 11:00:00.25 +01:00';
i interval year to month :=
  interval '1-4' year to month;
j interval day to second :=
  interval '2 01:02:03.04' day to second;
```

- Observaţie: În Oracle v8 şirul de caractere reprezentând data nu se prefixează cu numele tipului.

Evaluare expresii logice

- În cazul unei expresii logice complexe evaluarea se opreşte în momentul în care valoarea expresiei este cunoscută.
- Acest lucru înseamnă că restul expresiei nu se mai evaluează şi deci nici nu se generează excepţii în cazul în care acestea ar fi putut să apară în partea neprocesată a expresiei.

Exemplu

```
DECLARE
   a NUMBER;
   b NUMBER;
BEGIN
      .  .  .
   if (a = 0) or ((b/a) < 1) then
      .  .  .
   END IF;
      .  .  .
END;
```

- În cazul în care variabila **a** are valoarea 0 expresia logică se evaluează la TRUE şi nu se mai testează a doua parte a ei care ar genera o excepţie de tip ZERO_DIVIDE.

Valori nule (NULL)

- Comparaţiile care implică o valoare nulă returnează NULL.
- Negarea unei valori de NULL returnează NULL.
- O condiţie care se evaluează la NULL e tratată ca FALSE.
- În cazul expresiilor CASE (prezentate mai jos) nu se poate scrie WHEN NULL ci sintaxa este WHEN expresie IS NULL.

Valori nule (NULL)

- Un şir de caractere de lungime 0 este tratat de PL/SQL ca o valoare de NULL.
- Operatorul de concatenare || ignoră valorile nule.
- Operatorul IN (listă de valori) ignoră valorile nule din listă
- Operatorul NOT IN (listă de valori) întoarce FALSE dacă lista conţine o valoare nulă.

Valori nule (NULL)

- Când o valoare de NULL este transmisă ca parametru unei funcţii SQL sau PL/SQL rezultatul este NULL în afară de cazurile următoare:
- Funcţia DECODE aflată într-o cerere SQL poate conţine argumente nule
- Funcţia NVL poate avea primul argument NULL
- Funcţia REPLACE poate avea al doilea argument NULL (întoarce primul argument indiferent de al treilea) sau al treilea argument NULL (elimină din primul argument apariţiile celui de-al doilea)

Expresii CASE

- Expresiile CASE sunt folosite pentru a returna o valoare în funcţie de una sau mai multe condiţii.
- Ele suplinesc faptul că în instrucţiunile PL/SQL nu se poate folosi funcţia DECODE (aceasta poate fi prezentă doar în cereri SQL).
- Există două forme ale acestora. Prima dintre ele este exemplificată în continuare:

Expresii CASE

```
DECLARE
   codjudet VARCHAR2(2) := 'CJ'; numejudet
  VARCHAR2(20);
BEGIN
numejudet := CASE codjudet
WHEN 'MM' THEN 'Maramures'
WHEN 'CT' THEN 'Constanta'
WHEN 'CJ' THEN 'Cluj'
ELSE 'Alt judet'    END;
END;
```

Expresii CASE

- A doua formă este o generalizare a primei forme permiţându-ne ca valoarea returnată să fie aleasă în funcţie de o condiţie generală şi nu doar de egalitate cum este în primul caz.

Expresii CASE

```
DECLARE
codjud VARCHAR2(2) := 'CJ'; numejud VARCHAR2(20);
BEGIN
numejud :=
CASE
WHEN codjud = 'MM' THEN 'Maramures'
WHEN codjud ='CT' THEN 'Constanta'
WHEN codjud ='CJ' THEN 'Cluj'
WHEN codjud LIKE 'A%' THEN 'Nume incepand cu A'
WHEN codjud IS NULL THEN 'Cod judet NULL'
ELSE 'Alt judet'
END;
END;
```

Ce urmează?

- Săptămâna viitoare vom învăţa cum se folosesc :
  - Instrucţiunile de decizie
  - Instrucţiunile de ciclare
  - Instrucţiunile de alegere (CASE)
  - Folosirea etichetelor

Sfârşit